

AI-Powered ATS Scoring & Parsing Engine

Project Blueprint & Integration Guideline

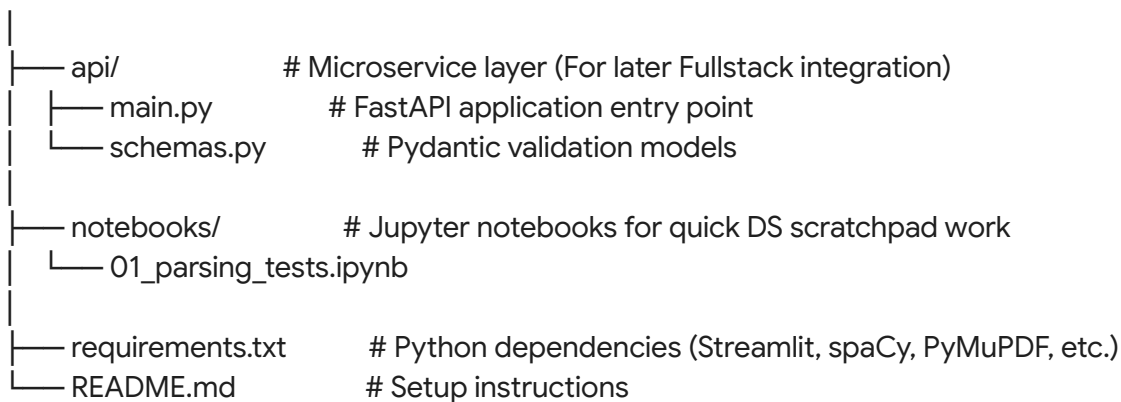
This document serves as the master guideline for building a Hybrid ATS (Applicant Tracking System) Engine in a pure Python environment. To accelerate development and visualize the ML models, the initial prototype will be built using **Streamlit**. Later, this core logic will be wrapped in FastAPI to be handed off to a full-stack developer as a microservice.

1. Project Folder Structure

This structure separates your data assets, your ML logic, the Streamlit prototype, and the eventual production microservice. It also includes an environment file for any API keys or model configurations.

ats-ai-engine/

```
|
|— .env                # Environment variables (e.g., API keys, model paths)
|— app.py              # Streamlit Application (Your interactive UI prototype)
|
|— data/               # Core data folder for local testing and parsing
|   |— resumes/        # Place sample .pdf and .docx candidate files here
|   |— job_descriptions/ # Place sample .txt or .json JDs here
|   |— taxonomy/       # (Optional) CSV/JSON of standardized tech skills
|
|— src/                # Core Python Modules (The "Engine")
|   |— __init__.py
|   |— config.py        # Hyperparameters (loads .env variables, alpha weights)
|   |— parser/          # Step 1: Document Ingestion
|       |— __init__.py
|       |— pdf_extractor.py # PyMuPDF/pdfplumber text extraction
|       |— docx_extractor.py # python-docx text extraction
|   |— nlp/             # Step 2: Normalization & NER
|       |— __init__.py
|       |— cleaner.py    # Text normalization, stop-word removal
|       |— entity_tagger.py # spaCy / HuggingFace NER extraction
|   |— scoring/         # Step 3: Math & Evaluation
|       |— __init__.py
|       |— sparse_bm25.py # Keyword exact matching (BM25)
|       |— dense_vector.py # Semantic matching (SentenceTransformers)
|       |— hybrid_engine.py # Weighted combination logic
```



2. Pipeline / Data Flow (Streamlit Prototype)

As a Data Scientist, you will build the logic for steps 1-4, visualizing everything directly in app.py.

1. Ingestion via Data Folder or UI:

- **Input:** You either upload a PDF/DOCX via Streamlit's `st.file_uploader`, or the app automatically loops through files in the `data/resumes/` folder.
- **Process:** Detect file type and route to `pdf_extractor.py` or `docx_extractor.py`.
- **Output:** Streamlit displays the raw extracted text so you can verify the parser is reading columns correctly.

2. Preprocessing & NER (Structuring the Text):

- **Process:** Clean text and run a pre-trained NER model (e.g., `spaCy`) to extract skills.
- **Output:** Streamlit visually highlights extracted entities (Skills, Years of Experience) using `st.write` or `spaCy`'s visualizer.

3. Sparse Retrieval (Keyword Scoring - BM25):

- **Process:** Tokenize the target JD from `data/job_descriptions/` and the parsed Resume. Calculate the BM25 score.
- **Output:** Streamlit outputs the Sparse Score.

4. Dense Retrieval (Semantic AI Scoring):

- **Process:** Use `SentenceTransformers` to encode the JD and Resume. Calculate Cosine Similarity.
- **Output:** Streamlit outputs the Dense Score.

5. Hybrid Aggregation & Dashboard:

- **Process:** Use a Streamlit slider (`st.slider`) to let you dynamically adjust the α weight: $(\alpha * \text{Sparse}) + ((1 - \alpha) * \text{Dense})$.
- **Output:** A beautiful dashboard showing the final percentage match and a breakdown of why the candidate scored that way.

3. Recommended Python Stack

Your `requirements.txt` should contain the following libraries to support a pure Python

environment:

- **UI Prototyping:** streamlit
- **Configuration:** python-dotenv (to load variables from the .env file)
- **Parsing:** PyMuPDF (fitz) for PDF layout parsing, python-docx for Word.
- **NLP/Text Cleaning:** spaCy, nltk.
- **Sparse Scoring:** rank_bm25.
- **Dense Scoring:** sentence-transformers, scikit-learn.
- **API (For later phase):** fastapi, uvicorn, python-multipart.

4. Phased Development Plan

Follow these steps to build the engine iteratively:

- **Phase 1: Data Setup & Parsing (The Foundation)**
 - Populate the data/resumes/ folder with 5-10 varied formats (multi-column, single-column, tables).
 - Build the src/parser/ modules. Use Jupyter notebooks first to ensure your parsing logic can extract clean strings without merging columns together inappropriately.
- **Phase 2: Streamlit Prototyping (Visual Debugging)**
 - Create app.py.
 - Build a simple UI to upload a resume or select one from the data/resumes/ folder.
 - Print the parsed text to the UI to verify it works visually.
- **Phase 3: NLP & Scoring Engine**
 - Integrate spaCy, rank_bm25, and SentenceTransformers into the src/ modules.
 - Update app.py to calculate and display the match scores against a sample JD from data/job_descriptions/.
 - Add interactive sliders to test different weighting mechanisms.
- **Phase 4: API Wrap (Full-stack Prep)**
 - Once the Streamlit app proves the logic works flawlessly, wrap the src/ modules into a FastAPI endpoint (api/main.py).
 - Define strict JSON inputs and outputs.
- **Phase 5: Handoff**
 - Dockerize the application. Provide the endpoint documentation to your Full-stack Developer so they can connect it to React/Next.js and a backend database.